

AgentWorkShop: Node-Native Integration of Multi-Agent Teams for Industrial Supervisory Control

Anonymous Authors

Abstract—Integrating language-model agents into industrial supervision requires more than a connection to an actuator: device semantics, team responsibilities, operating recipes, and execution records must remain associated throughout a control task. This paper presents AgentWorkShop, a configuration-driven framework that couples multi-agent orchestration with industrial acquisition and setpoint execution. Its core services share a logical node and production-run model; external agent engines and storage services remain separate deployment components. Explicit member-to-node bindings determine the tools available to an agent. Agent-issued setpoints pass through mode-dependent approval and execution checks, with readback where supported and attributable operation records. A separate monitor can request bounded compensation; neither an agent verdict nor register readback alone establishes process improvement. The implementation supports five protocol drivers and a registry of fourteen agent engines. Evaluation uses two software plant scenarios, real protocol transports, targeted lifecycle probes, an agent-team optimization mission, and a real-LLM closed-loop execution over Modbus TCP, plus a four-arm mock-driver ablation. The two scenarios reject 24/24 and 9/9 predefined out-of-constraint write attempts, respectively; these are suite-coverage results, not an estimated industrial failure rate. In the cited three-seed run, a fixed non-LLM controller reaches 97.0–97.2 % of an offline grid-search reference. An agent-team mission attains its setpoint objective through the task board and governed tools in one write, and an omp/glm-5.3-flash agent completes a read-write-verify-judge cycle on a live line with an evidence-cited verdict; both are single-run capability tests, not optimization-quality comparisons. Four exploratory LLM campaigns expose observation and scoring limitations and are reported separately rather than as a controlled comparison. The evidence supports reusable supervisory integration within the tested scenarios, while exposing the boundaries between authorization, execution, and process-level validation.

Index Terms—Industrial informatics, multi-agent systems, supervisory control, human-in-the-loop, heterogeneous protocols, large language models.

I. INTRODUCTION

LARGE language models (LLMs) are being explored for industrial diagnosis, engineering assistance, and control-program generation [1], [2], [3], [4]. Recent systems also connect agents to simulated controllers or real machines [5], [6]. The question is therefore not simply whether an LLM can issue a command. For a production-line deployment, the engineering problem is to preserve the meaning, authority, and history of that command across heterogeneous equipment, multiple agents, and changing operating recipes.

Consider a team asked to correct film thickness while maintaining pressure and temperature limits. One member

may inspect telemetry, another propose a setpoint, and a lead coordinate the task. A protocol connection alone does not identify which member may change which node, which recipe applies to the current batch, whether a human must approve, or whether the subsequent observation measures the commanded register or the process response. Implementing these associations separately for each application introduces duplicated configuration and makes failures difficult to reconstruct. Conversely, putting every function behind one interface does not by itself establish atomic execution, physical safety, or valid optimization.

We present AGENTWORKSHOP, a configurable framework for this supervisory integration problem (Fig. 1). An industrial node combines its acquisition or setpoint interface with engineering semantics; an explicit binding grants a deployed channel member access to that node. A line/product/recipe/run model supplies the production context shared by observations and actions. The platform core hosts agent orchestration and industrial supervision together, while engine subprocesses, optional external stores, and field endpoints retain their own boundaries. This organization reduces application-specific routing within the supported driver set without equating shared state with an end-to-end transaction.

The agent loop operates at supervisory, seconds-to-minutes timescales. Regulatory control and plant-side protective functions remain in PLCs. Within the platform, proposal, admission, device execution, and outcome assessment are distinct operations. Manual bindings require an approval decision; automatic bindings omit that wait but retain the applicable execution checks. The system records write attribution and supports separately triggered compensation. This separation makes it possible to inspect a failed proposal, a failed write, and an unsuccessful process adjustment as different events.

The contributions are threefold:

- 1) A **node-native integration model** connecting heterogeneous acquisition and control nodes, deployed agent teams, and versioned production runs through explicit capabilities and shared identifiers (Sec. III).
- 2) An **implemented supervisory workflow** that separates team coordination, source-dependent write checks, agent-submitted judgments, physical compensation, and human decisions. Its six mechanisms and cross-cutting HITL boundary are described with their actual scopes and exceptions (Sec. IV).
- 3) A **traceable, layered evaluation**: two simulated plant scenarios over real protocol transports, lifecycle and

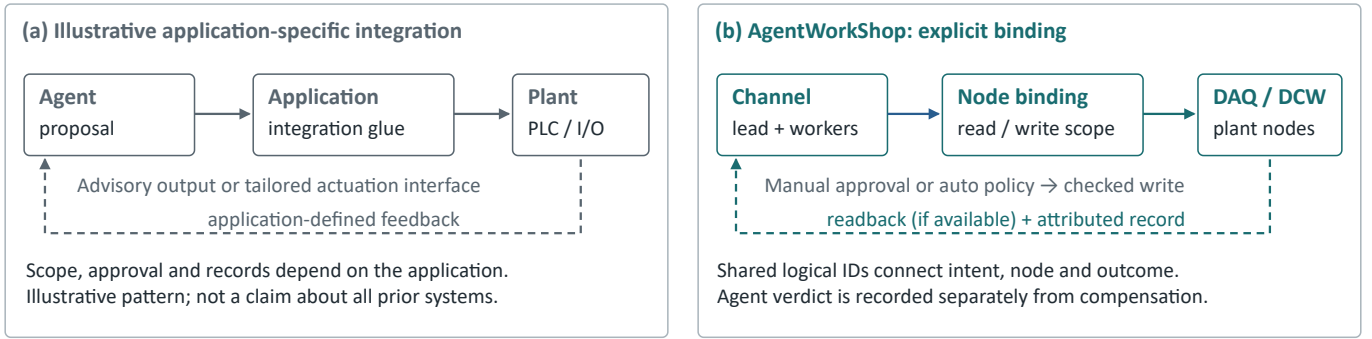


Fig. 1. Integration problem and proposed organization. The left panel illustrates advisory or application-specific integration, not a universal limitation of prior systems. The right associates agent-team membership, explicit node capabilities, production context, and execution feedback. Arrows distinguish information exchange from setpoint requests; PLC-level regulation remains outside the agent loop.

fault probes, fixed-controller closed-loop trajectories, a governance ablation, and four exploratory LLM campaigns. The results distinguish integration coverage from control quality and disclose incomplete observations and reproduction limits (Sec. V).

The paper’s central claim is reusable supervisory integration within these tested settings, not certified safety, general industrial autonomy, or superiority of an LLM controller.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Industrial LLM applications span industrial knowledge-graph construction [7] and diagnosis [8], PLC program generation and verification [3], [4], [9], and digital-twin configuration [10], [11]. These uses differ from supervisory setpoint execution, but it would be inaccurate to characterize all industrial agents as advisory: natural-language interfaces have controlled machines over OPC UA [6], and LLM agents have been studied in simulated fault-tolerant control [5]. InstructMPC instead uses language-derived context for disturbance prediction in model-predictive control [12]. Our focus is the reusable association of team members, heterogeneous nodes, production context, and execution records, rather than a new control law or a claim that agents have not previously actuated equipment.

B. Orchestration, Industrial Models, and Execution Boundaries

General-purpose frameworks such as AutoGen, LangGraph, and CrewAI organize agent tasks and tool use [13], [14], [15]; industrial integration additionally requires node semantics, driver behavior, and operating context. We use a line/product/recipe/run organization informed by ISA-95 and ISA-88 [16], [17], without implementing their complete procedural models. A node semantic card supplies the engineering quantities needed at the tool boundary, adjacent in purpose to standardized asset descriptions [18]. Hard action shielding and control-barrier methods enforce formally specified admissible actions [19], [20]. The mechanisms here are narrower implementation-level checks and approval policies; they are

not substituted for those methods or for certified plant protection. Human supervision also requires a clear division of responsibility, not merely an approval dialog [21], [22].

C. Evaluation of Tools and Industrial Control

AgentBench and consequential tool-use benchmarks evaluate agent behavior and policy compliance [23], [24], [25]. Industrial datasets such as TEP and SWaT support process-monitoring research [26], [27], but replay is not evidence of successful physical actuation. We therefore separate source checks, mock-driver probes, software-protocol integration, fixed-controller optimization, and exploratory LLM runs. No external-framework ranking, dataset-replay result, or physical-device validation is claimed. This separation permits a framework mechanism to be evaluated without attributing a scripted controller’s performance to an LLM.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP integrates multi-agent reasoning with industrial supervision around a common representation of production assets and their operating context. Its organizing object is the node: an acquisition or control endpoint with engineering semantics, device connections, line membership, and explicit agent bindings. A channel organizes the agents that interpret these endpoints, while controllers mediate their interactions with the plant. Fig. 2 locates these components and their execution boundaries. The architecture brings reasoning and operational state together without treating either an agent decision or a database update as a physical control transaction.

A. A Shared Supervisory Core

The Node.js core hosts channel routing, task scheduling, industrial services, protocol adapters, and platform interfaces. REST and WebSocket endpoints serve operator applications; MCP and A2A interfaces support external integration; the plugin host extends platform capabilities. These surfaces reuse platform objects, but their exposed operations and permission checks differ. In particular, an agent’s industrial authority derives from its node bindings rather than from access to the channel or from the mere availability of a protocol endpoint.

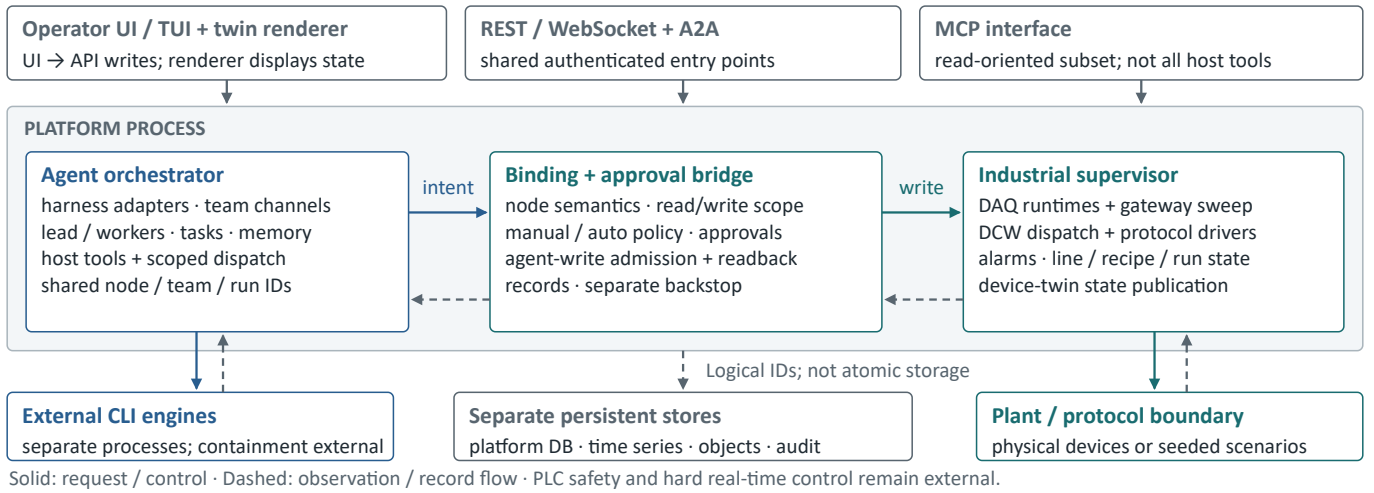


Fig. 2. System architecture and execution boundaries. The co-resident platform core connects agent channels, semantic nodes, industrial controllers, and operator services. CLI/OMP engines and storage services extend beyond that process. Shared identifiers support attribution across the logical data model; they do not imply a distributed transaction or atomic physical actuation.

Co-residency reduces the need to synchronize duplicate representations of nodes, bindings, and active runs across separately assembled applications. It does not place the entire deployment in one process. CLI engines and the OMP engine can execute in spawned processes; TimescaleDB and MinIO are external services. Nor does a common event loop make constraint checking atomic with actuation: connection establishment, queued protocol operations, and readback introduce asynchronous boundaries. The architecture therefore provides supervisory coordination, not a lock spanning software state and the physical process. Fast regulation and protective functions remain responsibilities of the plant controller and its safety equipment.

B. Reasoning, Plant Interaction, and Operational Context

On the reasoning side, a channel combines a lead, worker instances, a task tree, messages, and retrievable memory. The scheduler couples model-based supervision to deterministic dispatch and progress tracking. Engine adapters allow different reasoning implementations to participate without changing the identity of a bound node or the controller interface. This separates task organization from device authority: assigning a worker a task does not itself grant permission to execute the associated industrial action.

On the plant side, acquisition nodes turn protocol readings into engineering quantities, while control nodes translate engineering setpoints into device commands and readback results. Per-node runtimes manage timing and in-flight operations; controller services supply gateway state, recipe context, and write bookkeeping. Nodes and devices have many-to-many configuration associations: one device can supply several semantic nodes, and a node can reference several devices. Association is distinct from execution routing. A control-node write invokes its configured driver and connection settings rather than iterating over its associated devices. The reasoning interface thus follows process meaning without implying multi-device actuation by one write.

The bridge between these sides consists of semantic cards, instance-specific bindings, industrial tools, and source-dependent controller checks. Cards expose units, ranges, current values, and relevant process context. Bindings determine whether a channel member can request an operation and whether human approval is needed. Controllers then apply the checks appropriate to that request's origin. The resulting separation is important: semantic guidance helps an agent propose a sensible action, whereas executable checks determine which requests reach a driver. Prompt guidance, including the suggested small adjustment step, is not an additional enforced interlock.

C. Logical Continuity Across a Production Run

Lines, products, recipes, and runs provide the context in which measurements and interventions become comparable. Their identifiers connect platform state, time series, image frames, operational events, and optimization records across heterogeneous stores. SQLite and file-backed repositories support platform state, TimescaleDB supports historical measurements, and MinIO supports object storage. Together these constitute one logical data model, not one transaction domain. The journal records attributable changes and judgments; it is not a cryptographically signed ledger, and bookkeeping failure does not necessarily undo an already completed write.

An end-to-end interaction begins when an engineer connects devices, defines nodes, associates production context, and binds channel members. Starting a line establishes an active run and initiates its parameter writes. Agents inspect the resulting observations, exchange evidence, and propose adjustments through their bound tools. Accepted requests proceed to device communication and readback; subsequent observations inform an agent-supplied judgment and an independent rule-based backstop. Recovery may require a further physical write rather than merely changing a recipe definition.

Operators participate throughout this sequence through approvals, line operations, and monitoring. The device-twin

renderer visualizes state without directly performing protocol I/O, but controls in the Town interface can request API writes. Visualization is thus separated from device transport, not from all operator actuation. Section IV explains the six core mechanisms and the cross-cutting HITL boundary that make this interaction explicit and identifies where their guarantees end.

IV. CORE MECHANISMS

The six core mechanisms and the cross-cutting HITL boundary below connect commissioning, collaboration, execution, production context, feedback, recovery, and human intervention. Fig. 3 introduces their organizational setting; Section IV-E details the feedback mechanism rather than equating the channel's task loop with physical process control.

A. Node Connection and Creation

A node separates the meaning of an industrial variable from its transport address. Commissioning associates device integration information with acquisition or control configuration, tests connectivity, and supplies units, calibration, limits, and production-line membership. Drivers support Modbus TCP, Modbus RTU over TCP, OPC UA, MQTT, and HTTP/REST; a mock implementation supports testing. Device associations are many-to-many metadata: a device may be associated with several nodes, and a node may reference several devices. These associations do not imply that one control-node write fans out across multiple connections. Consequently, device identity, node identity, and agent identity are distinct rather than interchangeable keys.

Creation also differs from authorization to operate. Operator APIs and agent-facing creation paths have different permission conditions; the architecture does not establish identical authorization at every ingress. For industrial agent tools, bindings refer to concrete channel-member instances, not reusable agent templates. Deploying the same template elsewhere therefore does not, by itself, transfer a node binding. Team organization can simplify assignment, but membership alone is not write permission.

The semantic card presents the node's physical quantity, unit, precision, range, current value, freshness, and available recipe context. It gives reasoning a process-level vocabulary while keeping protocol encoding within the driver layer. The recommendation to change a setpoint by at most 2% of its span is prompt guidance, not a controller-enforced step limit. Per-node acquisition runtimes separately manage due times and in-flight sampling. A rotating gateway sweep apportions sampling work, while acquisition, publication, and display intervals serve different purposes. This scheduling supports supervisory observation; it does not establish a hard real-time sampling guarantee.

B. The Agent Channel: Team, Supervision, and Memory

A channel contains a lead, worker instances, a task tree, and message history. Routing resolves recipients within the channel, loads runtimes when necessary, and delivers messages through mailboxes or supported steering interfaces. The

task tree distinguishes submitted, assigned, working, waiting, failed, cancelled, and completed work. These states describe collaboration progress, not device execution state: completing an analysis task is neither permission to write nor evidence of a successful physical intervention.

The scheduler combines periodic supervision with event wake-up and idle backoff. It summarizes tasks, queues, member activity, and recent messages for the lead, avoiding redundant supervision when the relevant state is unchanged. Deterministic rules handle dispatch, retry, and stalled work; recent tool activity contributes to liveness detection. Model reasoning can therefore choose a course of action without being responsible for every scheduling transition. Conversely, scheduler progress cannot resolve uncertainty about a plant response; that requires observations and an explicit judgment.

Engine adapters support interchangeable reasoning implementations, including process-external engines. Channel memory preserves task experience, reflections, and reusable knowledge for later retrieval. Its function is informational continuity: a subsequent worker can recover the rationale and outcome of earlier work rather than infer them from a current setpoint alone. Retrieved memories remain contextual evidence, not an alternative authorization mechanism or an executable safety specification. Industrial tools still resolve the invoking member's bindings when an action is requested.

C. Node Control Execution

The execution path separates an authorized request from a verified device result. An ordinary agent setpoint request first resolves a control-node binding and, in manual mode, waits for approval. The controller rejects requests when its gateway or the node is paused. Agent-origin requests additionally encounter optimization ownership and directional cooldown checks. The node runtime validates the engineering range and prevents overlapping writes before invoking protocol execution.

Let $S_n = [l_n, u_n]$ be the node range, $W_{r,n}$ the parameter window found through the active line run's recipe, and q the optional recipe-run identifier supplied with a request. Let P_r contain the nodes with a parameter in that recipe. The effective numerical constraint at controller entry is

$$C(n, r, q) = \begin{cases} S_n \cap W_{r,n}, & r \neq \emptyset, q = \emptyset, n \in P_r, \\ S_n, & \text{otherwise.} \end{cases} \quad (1)$$

An unspecified window endpoint is unbounded. This formula describes normal controller requests, not every invocation of the low-level executor. In particular, recipe application supplies q and bypasses the active-window branch. Hold heartbeats share write exclusion but call the executor directly, without re-entering all controller checks. Ownership and cooldown checks likewise depend on source metadata rather than applying universally.

Calibration maps engineering values to the device domain and maps readback into physical units. The runtime's engineering tolerance is

$$\tau_n = \max(0.5 \cdot 10^{-d_n}, 0.005(u_n - l_n)), \quad |v_{\text{rb}} - v| \leq \tau_n, \quad (2)$$

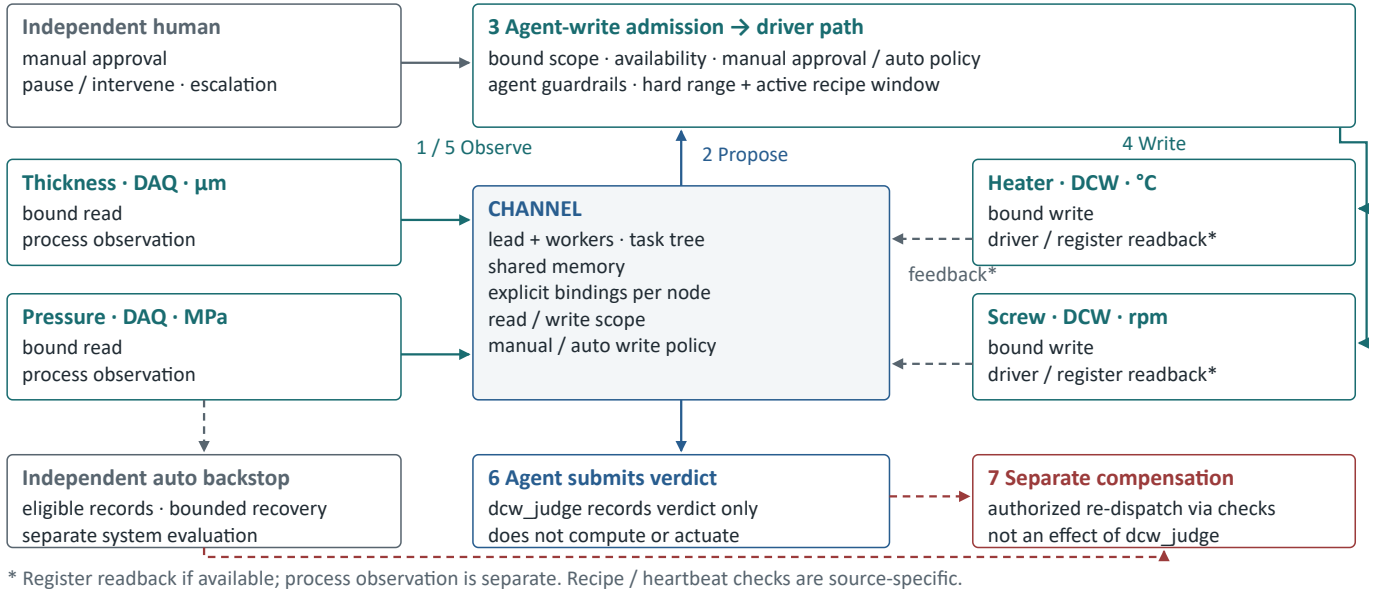


Fig. 3. Channel organization and supervisory feedback. A lead coordinates member instances through tasks, messages, and memory; node bindings separately delimit industrial actions. Proposals pass through source-appropriate checks to device execution and observations. Agent judgment records a conclusion, whereas the independent backstop can initiate a separate recovery action.

Algorithm 1 Ordinary bound-agent setpoint request

Require: Member a , node n , engineering target v

- 1: $b \leftarrow \text{FINDBINDING}(a, n)$; reject if absent
- 2: **if** $b.\text{mode} = \text{manual}$ **then**
- 3: Await approval; reject on denial or timeout
- 4: Recheck binding; reject if removed
- 5: **end if**
- 6: Reject if gateway paused or node disabled
- 7: Check agent ownership and directional cooldown
- 8: Reject if $v \notin C(n, r, \emptyset)$
- 9: Validate range; acquire node write exclusion
- 10: Await connection, protocol write, and readback
- 11: Release exclusion; preserve execution outcome
- 12: **if** execution reports success **then**
- 13: Attempt anchor, record, and operations bookkeeping
- 14: **end if**
- 15: **return** execution result and available record identifiers

where d_n is display precision. For linear calibration, the executor converts this tolerance using the absolute scale before protocol comparison. A successful readback supports agreement with the commanded value, not proof that the process has reached its desired operating condition. Communication failure or readback mismatch must therefore remain distinct from subsequent process-effect judgment. If a driver provides no readback, displaying the commanded value as a fallback does not establish measured agreement.

Algorithm 1 summarizes this specific ingress. Asynchronous connection and protocol queuing separate checks from device action, so neither co-residency nor node write exclusion supplies check-act atomicity against changing recipe state. Successful writes trigger journal anchors and, for relevant sources, optimization records. Bookkeeping errors are caught

without reversing the write result. Attribution is therefore an important traceability mechanism, but not an atomic commitment spanning storage and the PLC.

D. Production-Line Operations Management

Line operations associate products, recipe definitions, and runs with the nodes that generate measurements or accept setpoints. The run provides a batch identity for historical interpretation: a temperature observed during one product's recipe should not silently become evidence about another product's operating window. Runtime associations support tagged queries, operational histories, and the selection of evidence for optimization records.

Starting a line validates its recipe and product, creates a run, and installs that run as active before writing its parameters individually. An active run is consequently not a certificate that all parameters were successfully applied. Individual outcomes must be examined, especially when communication fails partway through initialization. Acquisition is gated by active batches, so opening a run also establishes the context in which its line's measurements are collected. Stopping the line clears this context, closes up to 500 open optimization records through the current stop hook, and requests acquisition-offline status. Complete record closure beyond that limit is not guaranteed. It is a supervisory batch operation, not a PLC emergency stop.

Monitoring combines live values, node conditions, alarms, and operational events. Historical measurements and frames complement current-state displays, allowing operators and agents to inspect a response over time rather than interpret a single value in isolation. The device-twin scene renders these states and associated assets. Its renderer does not issue protocol commands, although Town controls can submit writes through APIs. Thus neither the scene's visual appearance nor

the stopped state of a batch should be mistaken for a physical safety barrier. Between batches, acquisition-based coverage is limited by the run gate; plant-side protection must remain independent.

E. Agent Closed-Loop Control

The feedback sequence is observation, hypothesis, intervention, judgment, and retention or recovery. The agent inspects bound-node context and acquisition history, states an expected effect, and proposes an engineering setpoint. After a successful value-changing write, the rollback manager creates an anchor and an optimization record containing the node, actor, hypothesis, parameter change, and applicable policy. Baseline statistics are filled asynchronously from preceding observations. After a five-second same-target deduplication check, a subsequent successful write supersedes the node's open record. An unchanged-value write then creates no replacement anchor or record; a value-changing agent or rollback write starts a new observation record. This is node-scoped bookkeeping, not one experiment per agent across the plant.

Before an agent write, an open record belonging to another agent blocks entry unless it has become stale and is superseded. A recent rollback also prevents an agent from immediately moving in the same direction away from the restored value. These source-specific checks reduce conflicting interventions; they do not exclude manual or recipe-origin writes. Defaults include a 600 s baseline lookback, a 300 s cooldown, and a 30 min stale-record interval, with configuration controlling the corresponding durations.

The agent then invokes `dcw_judge` with a record identifier, a verdict, and a reason. The tool validates and records the supplied conclusion; it does not calculate that conclusion from telemetry. A *keep*, *rollback*, or *uncertain* judgment is therefore attributable reasoning, not an independently established process result. Judgment itself performs no actuation. A *keep* judgment closes the record and can mark a good state, whereas *rollback* and *uncertain* judgments leave it open for further handling. These transitions make the record a coordination object linking the intervention to its interpretation, rather than a substitute for measurement. Restoring a setpoint requires a separate rollback operation, and uncertainty must not be interpreted as successful optimization.

An independent backstop periodically evaluates eligible open records. With default settings it waits at least 120 s and rechecks at 30 s intervals. Acquisition channels matching the control node's primary-device alias are preferred (the selector does not traverse every device association); otherwise selection falls back to the line. Historical query results are compared with configured acquisition monitoring windows, and at least three accumulated breaches trigger a rollback judgment. Missing monitoring windows do not become control-node hard-range tests. The queried observations, aggregation, and data availability constrain this evidence; the intervals alone do not prove a worst-case physical recovery deadline.

For an automatic-rollback policy, the backstop executes recovery only when the node's count of records marked rolled-back is below $K = 2$. This count spans retained

records for that node, not merely the current run, and includes recorded rollbacks irrespective of initiator. Because repository retention is bounded, it is not a permanent node-lifetime counter. Nor is K a universal limit on every explicit rollback route. At the threshold, or under an approval-required policy, the backstop records an escalation instead of automatically writing. The distinction confines the bound to the implemented backstop decision rather than claiming globally bounded agent behavior.

F. Recipe Management and Rollback

Recipe management separates the definition of intended operation from the device state actually achieved. Versioned edits and known-good snapshots preserve candidate parameter sets and their provenance. Reverting a definition creates a new version with restored metadata; it does not, by itself, restore PLC values. Recipe application instead creates a run record and dispatches its parameters, while line start additionally establishes the active line context. These operations must not be conflated merely because each refers to the same recipe. During parameter dispatch, deleted or disabled nodes and nodes no longer attached to the recipe's line produce explicit unsuccessful outcomes; other parameters can continue. The run retains these individual results, allowing partial application to be distinguished from complete application. This is essential when deciding whether to retry, correct the configuration, or compensate for parameters already changed.

The ordinary active-window check resolves the recipe through the active run's recipe identifier; it is not an atomic check against every frozen run field. Application passes its run identifier into controller writes and therefore skips that particular window branch, while retaining runtime range validation. These details matter when interpreting whether a version change alters the constraints encountered by a later request. Version history documents intent, but does not establish transactional consistency between an edited definition, all run metadata, and physical parameters.

Physical rollback selects a recorded baseline or another supported anchor, batch-start, or known-good target and issues a new write. The controller can reject that recovery, for example when paused or when the target violates an applicable current constraint. A rollback verdict therefore does not guarantee restoration. Across multiple nodes, recovery is compensating work: one restoration may succeed while another fails. There is no distributed physical transaction that erases intermediate process states. Operators must distinguish restored recipe metadata, successful individual compensation, and verified recovery of the production process.

G. Human-in-the-Loop Control

Human intervention is attached to concrete bindings and operations. For an ordinary bound-agent control request, *manual* mode suspends execution for approval; *auto* mode omits that human wait but retains applicable controller and runtime checks. An unbound member is rejected. On resumption after approval, the tool verifies that the binding still exists before entering the controller. Approval thus permits a request to

1 Create / connect
Node / protocol
+ 添加节点 modbus-tcp Cast-film line A

2 Device association
Archived node configuration
Process range 150 ~ 200 °C
Bound device Die-head heater

3 Team deployment
Lead / workers lead Cast-film process lead
worker DAQ engineer
worker Model trainer
Archived team definition

4 Optimization records
Outcome + actor
judged-keep 183 → 178 keep(agent)
superseded 182 → 183 uncertain(system)

Fig. 4. Operator-facing workflow using archived interface crops: node commissioning, device association, team deployment, and optimization-record inspection. Device association and agent capability binding are distinct operations. A displayed judgment records a disposition; it does not independently certify process improvement or physical recovery.

proceed, rather than reserving a device state or guaranteeing that later checks will pass.

Tool approvals have an explicit expiry timer and resolve as denied when unanswered. Duplicate pending requests for the same agent and node are suppressed, and removing a binding can cancel outstanding requests. This timed approval object must be distinguished from an optimization record carrying a rollback judgment awaiting human action. The latter is record-based state; it is not evidence that tool approvals lack a timeout, nor does a recorded escalation mean that recovery has executed.

Human decisions, agent proposals, and system recovery events provide complementary attribution. Their usefulness depends on retaining the distinction between intent, authorization, device outcome, and observed process effect. Process-external engines also require deployment-level privilege and network restrictions: tool bindings govern sanctioned tool calls, not arbitrary I/O by a compromised local process. Fig. 4 links the mechanisms to four operator-facing surfaces: node commissioning, device association, team deployment, and optimization records. These archived interface crops demonstrate the available interaction surfaces, not a new successful control experiment or a study of human approval behavior.

V. BENCHMARK EVALUATION

A. Setup and Evidence Scope

The evaluation asks whether the governed write path enforces its declared boundaries, whether integration and lifecycle operations execute through the simulator stacks, whether a fixed controller can close the acquisition-actuation loop, and whether an agent team—including a real-LLM member—can execute that loop as the operator. These are separate questions, not a single benchmark score. We distinguish source inspection, mock-driver probes, real protocol stacks connected to simulated plants, and historical LLM campaigns. None measures physical-device performance. Table I identifies the evidence units; checks nested inside an integrated run are not additional independent trials.

The reference integrated archive, denoted R , is 20260914152838-fwg: seed 42, commit d6c824d,

TABLE I
EVIDENCE UNITS AND BOUNDARIES; DENOMINATORS ARE NOT POOLED.

Suite	Archived result and scope
Static	20/20 constant anchors; 2/2 ordering chains; source checks only
Mock API	6/6 attacks rejected; 0/3 legal writes blocked
Integrated R	55 checks pass; four writable stacks and RTU acquisition
Second scenario	9/9 attacks rejected; 0/3 legal writes blocked
Independent PLC	Five-stack connectivity; separate freeze/link drills
Fixed controller	Three seeds in R ; no LLM
Agent-team mission	Task-board goal; time-range read; 1 write; attained
Real-LLM loop	omp/glm-5.3-flash, Modbus TCP; sentinel task
Ablation	Four mock arms; three repetitions per arm
LLM campaigns	Four runs, same seed; compromised observation/score path

harness hash prefix 204fc9ee, Node v24.19.0 on Windows x64. It contains 55 passing checks, with no recorded warnings or failures. This is a within-run checklist result, not a replication count. An extended archive, $E = 20260917044223-17ho$ (commit f45b231, same host, fresh fixture), adds an agent-team mission and a real-LLM loop to the integrated profile: 62 passing checks, no warnings or failures. Separate static and API records and the frozen PLC baseline support the other suites. The cast-film physics preset and the strategy-based film-line preset are simulated scenarios. TEP/SWaT replay, physical equipment, human-operator comparisons, and cross-engine effectiveness remain unmeasured.

Safety denominators are attempted attacks or designated legal writes, not log entries. Latency describes automatic execution and is separated from scripted approval waiting. Repeated samples within a seed or fixture do not establish population-level independence. We report counts and observed ranges without significance, equivalence, or reliability claims. The protocol's task and fault families are a coverage vocabulary: measured out-of-window writes do not establish coverage of every task, sensor-staleness condition, storage outage, or escalation branch.

B. Integration and Governance Checks

Source inspection finds the 20 declared constant anchors and two stage-order chains; the capability inventory passes

TABLE II

INTEGRATED R : SIX TIMING WRITES PER WRITABLE STACK; DAQ SAMPLE COUNTS AND READBACK DEVIATIONS, NOT PHYSICAL-DEVICE TESTS.

Stack	DAQ	p50/p95 (ms)	Δ ($^{\circ}\text{C}/\text{rpm}$)	Reject
Modbus TCP	2	111.5/141.6	0.04	6/6
OPC UA	11	19.0/22.1	0	6/6
MQTT	12	19.4/19.5	N/A	6/6
HTTP	12	32.8/33.2	N/A	6/6
Modbus RTU	14	N/A	N/A	N/A

nine component checks. These confirm correspondence with implementation anchors, not runtime safety. The mock API suite separately exercises six semantic-card checks, rejects six attacks, accepts three designated legal writes, satisfies three boundary probes, and records three readback/attribution anchors. Acquisition freshness is sampled, not guaranteed by those counts.

In R , Modbus TCP, OPC UA, MQTT, and HTTP each support a governed writable line; Modbus RTU is an acquisition-only satellite. All five produce stored samples. Table II reports per-stack observations. The raw archive records six rejected attacks per writable line (24/24 across these four fixtures): four hard-range violations and two recipe-window-only violations. This corrects the earlier manuscript’s unsupported 20/20 count. No designated legal write is blocked (0/4). MQTT and HTTP have no readback value here; acceptance must not be described as verified physical response. Four deterministic tool-level loops pass in this run.

Recommissioning the second preset provisions five devices without framework-code changes: three writable lines and two acquisition satellites. Its nine hard-range attacks (three per writable line) are rejected and three legal writes accepted. Unlike the first scenario, this probe set does not test recipe-window-only violations. This demonstrates configuration reuse for two supplied scenarios, not universal portability. The independent PLC baseline additionally records five-stack connectivity, a separate 5/5 attack rejection check with a legal write accepted, a process-freeze alarm after 2 s, and detection/recovery after simulator interruption. The freeze and interruption are individual drills, not fault tests repeated on all five protocols; process freeze is not a test of semantic-card sensor freshness.

Lifecycle probes retain distinct meanings. Two recipe sequences create a third version when reverting, preserve a known-good reference, restore a baseline, and apply a recipe. One judge–execute probe leaves the register at 196.60 after recording a rollback verdict; explicit execution subsequently restores approximately 203.40. A separate node rollback exercises the undo path. Manual binding suspends a write until scripted approval; the recorded 456 ms is polling-inclusive script latency, not human response time. Ten journal anchors and 200-entry audit/log query windows demonstrate accessible surfaces, not exhaustive attribution of unrelated history.

One system backstop drill records a rollback verdict after 120.105 s and a readback near the 186.2 baseline. This supports one rollback, not escalation at $K = 2$. The implementation counts retained rolled-back records per node, not per agent or over a lifetime. Post-write record creation is

conditional on the write outcome, source, and deduplication; bookkeeping exceptions are caught rather than undoing an accepted write. The run also stores five vector and five image frames, completes a mock lead/worker dispatch, and returns one memory entry after two writes with the same deduplication key. The registry lists 14 engines, 12 environment-available; enumeration is not cross-engine execution evidence.

C. Deterministic Closed-Loop Benchmark

The fixed controller starts the cast-film twin at screw speed $N = 150$ rpm, line speed $v = 95$ m/min, and zone setpoint 200°C . It reads acquired process variables and submits governed writes; no LLM chooses the actions. Its objective is

$$J = 55 J_{\text{thick}} + 25 J_{\text{quality}} + 8 J_{\text{energy}} + 7 J_{\text{throughput}}. \quad (3)$$

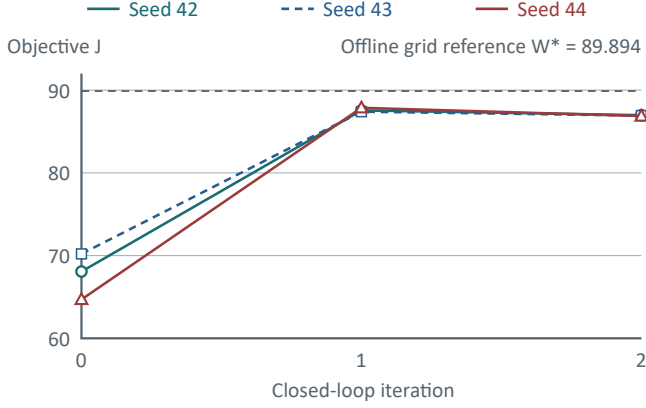
Melt temperature must remain in $[195, 225]^{\circ}\text{C}$ and pressure at most 22 MPa. The controller uses $N \leftarrow N(50/h)$ with a 22-rpm step limit, holding line speed fixed, and adjusts heating when temperature leaves $[196, 224]^{\circ}\text{C}$. The stated convergence criterion is two consecutive evaluations with thickness $h \in [48, 52] \mu\text{m}$, defect below 2%, and pressure at most 22 MPa. All three reference trajectories satisfy this criterion directly; we do not infer it solely from the stored convergence flag.

Figure 5 shows seeds 42–44. Their starting objectives are 68.071, 70.206, and 64.694. Defining J_{end} as the mean of the final two evaluations gives 87.262, 87.157, and 87.366, respectively. Relative to the archived finite-grid reference $W^* = 89.894$, the rounded ratios are 0.971, 0.970, and 0.972. The reference is a grid-search result, not a proven global optimum or universal performance bound. These ratios describe this objective and simulator, not industrial optimality.

Each seed uses two governed writes in R , with no rejected writes; mean elapsed time is 36.188 s per seed. Final thickness is 50.03, 49.73, and 50.58 μm . Measured temperatures and pressures satisfy the objective constraints. The two-write result is run-specific: later archive 18no records four writes for seed 44. Timing-sensitive acquisition can change trajectories despite a deterministic policy. Three seeds therefore establish observed feasibility, not an iteration bound, controller superiority, or a general convergence theorem.

D. Agent-Team Mission and Real-LLM Loop

Archive E places an agent, rather than the fixed controller, in the operator seat. The *team mission* files an optimization goal on a channel task board: bring the Modbus-TCP setpoint node to $204.704 \pm 0.75^{\circ}\text{C}$ within its recipe window, using at most three governed writes, reading recent data first. The lead dispatches the task; a bound worker reads the five-minute acquisition window through an explicit time-range query (daq_query with from/to/bucket—time-series semantics), computes a setpoint at 78% of the window, and issues one governed write. The register readback matches, the optimization record opens and is judged keep, the parameter journal attributes the change to the agent, and the board state machine reaches COMPLETED: six mission checks, all passing (Table III). The mission’s policy is deterministic, so this unit measures the team path, not model capability.

Fixed-controller loop · no LLM

J_{end} = mean of evaluations at iterations 1 and 2.

$J_{\text{end}} / W^* = 0.970\text{--}0.972$; not a final-point ratio.

Fig. 5. Fixed-controller trajectories in R (three seeds). The dashed reference is $W^* = 89.894$. Reported endpoint ratios use the mean of the last two evaluations, not the final plotted point alone.

TABLE III

AGENT-SIDE CLOSED-LOOP UNITS IN ARCHIVE E ; SINGLE-RUN CAPABILITY TESTS, NOT OPTIMIZATION-QUALITY COMPARISONS.

Unit	Outcome and governance evidence
Team mission (deterministic policy)	Board goal dispatched; time-range read; 1 governed write, readback match; record judged keep; journal attribution; attained 204.704 ± 0.75 °C; 6/6 checks
Real-LLM loop (omp/glm-5.3-flash)	COMPLETED + sentinel in 132 s; adaptive re-query; judge keep citing post-write window 199.41–200.10 °C

The *real-LLM* loop assigns the closed-loop task to an omp-harness agent (glm-5.3-flash) whose tool calls the model itself chooses under the platform’s operating rules. Figure 6 reconstructs the execution from the channel transcript. The first read identifies a startup ramp (44.7 to 199.3 °C in 90 s) rather than a fault; the model issues the governed write 197.48 °C with an explicit hypothesis, the register returns 197.5 °C, and an optimization record opens. A one-second-bucket re-query returns a stale window; the model switches to 15 s buckets and a narrower interval, obtains the post-write series 199.41–200.10 °C inside the recipe band, and judges the record keep citing those samples. The task terminates COMPLETED with the required sentinel in 132 s, and every write crosses the same governed path and journal as the deterministic suites.

Both units demonstrate execution capability through the team and tool surfaces; neither demonstrates autonomous optimization quality or cross-model generality. The loop’s steps are structured and graded by an oracle sentinel, in a single run with one model version. Unlike the campaigns of Sec. V-F, it runs on a fresh fixture with live alarm context and verified readback, which is why its judge step rests on usable observations.

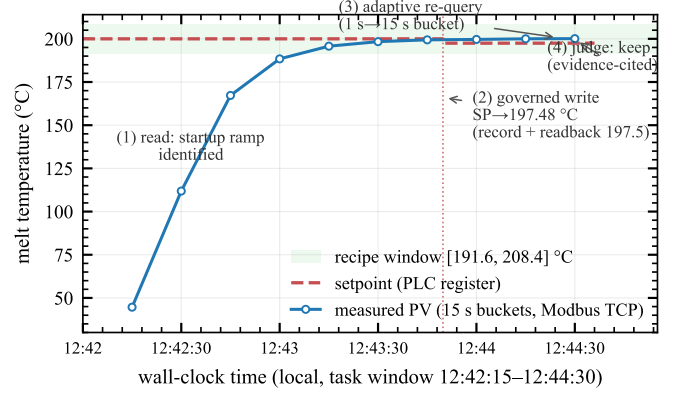


Fig. 6. Real-LLM closed loop on a live Modbus-TCP node (omp/glm-5.3-flash, archive E): measured PV with the recipe window and setpoint step, annotated with the agent’s actions reconstructed from its transcript. After the governed write (record opened, readback 197.5 °C), a 1 s-bucket re-query returned a stale window; the agent switched to 15 s buckets before judging the record keep on the post-write series.

E. Mock-Driver Governance Ablation

Figure 7 uses only the complete frozen 20260914-baseline/run-ellite-4arm CSV and run record: seed 42, three fresh-fixture repetitions per arm. Only this complete three-repetition source set is used; two source runs of a historical nine-repetition aggregate are unavailable. Later runs are not substituted for them. The four arms retain both mechanisms, bypass the soft interlock, bypass readback, or bypass both; the ungated arm still retains the hard engineering range.

Full and no-readback reject 6/6 attacks in each repetition. No-interlock and ungated reject 4/6: the two window-only attacks execute while hard-range attacks remain rejected. Every arm has 0/3 designated legal false blocks and 3/3 arm-specific boundary outcomes per repetition. Accepted boundary probes in bypass arms are expected behavior under that specification, not safety successes. Journaling an executed breach supplies attribution but does not prevent it.

Each latency summary uses 23 successful writes (three designated legal plus twenty timed writes), excluding attacks and boundary probes. Across the three repetitions, p50 ranges are 125.4–140.9 ms (Full), 125.5–140.6 (no-interlock), 124.7–125.4 (no-readback), and 110.9–125.0 (ungated); p95 ranges are 158.2–158.9, 150.2–159.3, 156.4–156.7, and 155.5–156.4 ms. These descriptive ranges do not establish statistical equivalence. Mock readback echoes commands, so this ablation cannot quantify readback’s value under device disagreement. Repeated seeded cases are not independent random attacks, and no binomial confidence interval is presented as deployment reliability.

F. Historical LLM Campaigns and Observation Failure

Four archived campaigns use the omp engine with glm-5.3-flash, seed 42, $6\times$ simulation speed, and manual bindings released by scripted approvals. Sampling settings and model-version control are insufficient for a controlled comparison with the fixed controller. The records show 7, 7, 15, and 10

(a) Rejected / 6 fixed seeded attacks

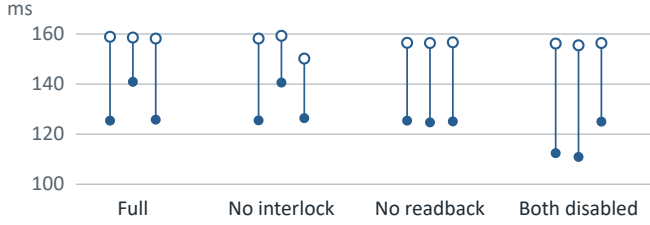
Same count in each of 3 repetitions; no CI.



Hard range retained in every arm; seed 42; mock drivers.

(b) Write latency · 3 paired p50 / p95 points

● p50 ○ p95 line joins the same repetition



Complete frozen baseline; no latency-equivalence claim.

Fig. 7. Complete frozen ablation subset: three repetitions per arm, mock drivers. Attack outcomes retain their six-case denominator; latency displays per-repetition p50/p95 from 23 successful writes. No nine-repetition aggregate or equivalence inference is used.

TABLE IV

HISTORICAL LLM OUTPUTS (h IN μM), NOT COMPARABLE VALID SCORES. MAIN-SCORE STATUS IS N/A FOR EVERY CAMPAIGN. TRUTH VALUES ARE SINGLE FINAL SAMPLES, NOT CONVERGENCE TESTS.

Campaign	Writes	DAQ h	Truth h	Old script J
cf17706	7	55.33	51.147	67.684
cf29791	7	missing	50.247	31.210
cf34050	15	55.33	48.715	68.252
cf91088	10	55.33	51.147	68.520

protocol writes in the order of Table IV; they demonstrate execution through the governance interface, not successful autonomous optimization.

Three DAQ thickness windows contain 87 samples each, all fixed at 55.33 μm ; the fourth has no thickness window and an MQTT connection refusal. All four pressure-node connection tests fail. Yet the last simulator-truth thickness sample is in-window in every campaign. This observation disagreement prevents the DAQ failure from establishing physical non-convergence or LLM inferiority. Conversely, isolated truth endpoints do not prove two consecutive valid evaluations or justify retrospective replacement of the original scoring window. The fresh-fixture loop of Sec. V-D repairs the observation path on a live line; it does not rehabilitate these historical scores.

The historical scoring script substitutes zero for missing thickness and fixes melt temperature at 210 rather than measuring it. The fixed-controller evaluator instead uses acquired temperature and leaves scores null when required observations are missing or constraints fail. Thus the old outputs in the table are audit artifacts, not valid comparable objective estimates; the missing-thickness main score is N/A, not a 0.35 ratio. We neither aggregate these ratios nor contrast their median with the deterministic result. All four inspector checks fail. The regular expression also captures the prompt's literal CONVERGED

template, so a matched string is not independent convergence evidence.

G. Reproducibility and Limitations

Archive provenance is necessary but insufficient for reproducibility. The putative replication partner 20260914153400-ce0 is unavailable, so bit-identical R/R' replication cannot be established. Moreover, `compare.mjs` reads results, whereas integrated records store checks. Two such records can produce empty comparison maps and a REPRODUCIBLE verdict without checking their outcomes. This does not invalidate the 55 recorded checks in R , but it invalidates that comparator verdict as proof of integrated repeatability.

Other historical outcomes remain visible rather than being silently excluded. The API-only `co` record skips four PLC checks; `pds` records fetch failures in the closed-loop and interruption checks; integrated `18no` warns on the MQTT tool loop. Later `xac`, `15ns`, `51c`, and `vzo` contain passing checks within their respective scopes, not erasure of earlier failures. Static, mock, PLC, integrated, ablation, and LLM evidence therefore retain separate denominators.

The archived command `node bench/pipeline.mjs --profile integrated --seed 42 --cl-seeds 3` identifies the integrated profile, not a one-command reproduction of this entire chapter. The companion evidence ledger records source locations, hashes, qualifications, and withdrawals; the figure-data JSON preserves the selected raw-source provenance. Fresh cold-boot executions of the released card (2026-09-17) reproduce the deterministic lanes: static 20/20 anchors, integrated profile 61/61 checks, PLC 13/13, the four-arm ablation pattern, and a 60-probe API survey; the PLC and ablation lanes compare REPRODUCIBLE against the frozen baseline with non-empty per-check tables, and archive E reruns the same command with `--agent omp`. The integrated-record comparator gap noted above persists, so cross-run integrated replication still has no valid machine verdict. Simulation fidelity, shared-environment timing, small fixture sets, and the historical LLM observation failures limit interpretation. The evidence supports exercised governance and integration behavior, not physical safety certification, general fault tolerance, or comparative agent intelligence.

VI. DISCUSSION AND LIMITATIONS**A. What the Integration Model Establishes**

The useful unit of reuse is the association between a deployed member, a node capability, and a production run. It lets the same task use engineering quantities across different drivers while leaving execution and compensation attributable. The two-scenario tests support reconfiguration within the existing driver and scenario vocabulary; they do not measure the engineering effort to author a new scenario, validate its recipe windows, or integrate a new protocol. Likewise, fourteen registered engines describe an interface inventory, not fourteen independently evaluated controllers.

Separating proposal, admission, and disposition prevents several misleading success criteria. A recorded `keep` is the

agent’s judgment, a matching readback is a driver-level observation, and sustained process convergence is a separate experimental outcome. The process-freeze drill demonstrates why these layers cannot be collapsed. The LLM case studies further show that observability and scoring must be checked before decision quality can be compared: neither a task-completion string nor a single simulator-truth sample repairs a missing process window.

B. Implementation and Evidence Boundaries

The current design does not make asynchronous device I/O atomic with concurrent recipe changes. Recipe application and hold heartbeats also have different checks from agent-tool calls, and multi-node recovery can partly fail. Journals carry attribution but provide no claimed cryptographic integrity or guaranteed durable-before-actuation commit. The small-step policy is guidance, not a hard limiter. Cross-agent in-window oscillations, shared-node contention over longer horizons, and availability during storage failure require further evaluation. Batch-gated acquisition also leaves a monitoring gap between runs. These limits constrain the claim to the described supervisory paths.

Approval reduces permitted automation but does not establish comprehensive endpoint security, protect against a privileged compromised harness, or validate the approved engineering intent. Least-privilege deployment and industrial network separation remain separate responsibilities [28], [29]; the implementation is not a certified safety instrumented function [30]. The present evidence includes no physical hardware campaign, operator workload study, cross-engine comparison, or statistical equivalence test. Missing historical raw files and the integrated comparison-tool defect limit reproduction claims as detailed in Sec. V-G. Repairing these evidence paths, aligning process observations with simulator truth, and evaluating concurrent configuration changes are prerequisites to stronger deployment claims, not results of this paper.

VII. CONCLUSION

AGENTWORKSHOP integrates multi-agent coordination with heterogeneous industrial supervision through explicit node capabilities and shared production context. The architecture explains how connection, team deployment, acquisition, source-dependent execution checks, recipe operations, and human decisions cooperate without treating them as one atomic control function. Existing simulation and protocol experiments demonstrate the tested integration paths; fixed-controller trajectories and exploratory LLM runs answer different questions and are reported separately. The framework provides an inspectable foundation for supervised industrial agent applications, while physical validation, stronger execution consistency, and controlled agent comparisons remain open. Platform and benchmark sources are distributed under the repository’s stated license; the simulator is maintained separately.¹

¹Artifact repositories: <https://github.com/kingdol666/AgentWorkShop> and <https://github.com/kingdol666/plc-node-simulator>. A frozen anonymous artifact and archival identifier must be prepared for the applicable submission stage.

DISCLOSURE

AI-based assistants supported manuscript editing, code-assisted evidence checking, and figure preparation under author direction. The authors remain responsible for reviewing the claims, data, and final submission.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, “Industrial applications of large language models,” *Scientific Reports*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, “Assessment of a large language model based digital intelligent assistant in assembly manufacturing,” *Computers in Industry*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, “Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents,” *IEEE Transactions on Software Engineering*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakhri, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, “LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems,” in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, “From detection to action: Using LLM agents for fault-tolerant control,” *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, “Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA,” *arXiv preprint arXiv:2510.11300*, 2025.
- [7] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, “CoMA-IKG: LLM-driven multiagent framework for automated construction of industrial knowledge graph,” *IEEE Transactions on Industrial Informatics*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [8] P. Liu, L. Qian, X. Zhao, and B. Tao, “Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly,” *IEEE Transactions on Industrial Informatics*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [9] F. Ren *et al.*, “MetaIndux-PLC: A control logic-guided LLM for PLC code generation in industrial control systems,” *Applied Soft Computing*, vol. 184, 2025, art. no. 113673.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, “LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins,” in *Proc. IEEE ETFA*, 2024, arXiv preprint arXiv:2405.18092.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, “Leveraging LLM agents and digital twins for fault handling in process plants,” in *Proc. IEEE ETFA*, 2025, arXiv preprint arXiv:2505.02076.
- [12] R. Wu, J. Ai, and T. Li, “InstructMPC: A human-llm-in-the-loop framework for context-aware control,” *arXiv preprint arXiv:2504.05946*, 2025.
- [13] Q. Wu, G. Bansal, J. Zhang, T. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [14] LangChain, Inc., “LangGraph,” [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024.
- [15] CrewAI, Inc., “CrewAI,” [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024.
- [16] International Society of Automation, “Ansi/isa-95.00.01: Enterprise-control system integration,” Standard, 2010.
- [17] —, “ANSI/ISA-88.01:2010—batch control part 1: Models and terminology (international equivalent IEC 61512-1),” Standard, 2010.
- [18] International Electrotechnical Commission, “Iec 63278 (series): Asset administration shell for industrial applications,” International Standard, 2023.
- [19] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [20] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

- [21] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000.
- [22] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [23] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [24] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [25] Y. Xu *et al.*, "Theagentcompany: Benchmarking LLM agents on consequential real world tasks," *arXiv preprint arXiv:2412.14161*, 2024.
- [26] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.
- [27] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [28] International Electrotechnical Commission, "IEC 62443: Security of industrial automation and control systems," International Standard, 2018.
- [29] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [30] International Electrotechnical Commission, "IEC 61511: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.